

SQL Notes for DBMS Placements

SQL (Structured Query Language) is used to store, query, update, and manage relational data. For placements, focus on writing correct queries, understanding joins and grouping, and explaining what happens logically inside a query.

1. Basic SELECT

SELECT is used to retrieve data from one or more tables.

```
SELECT column1, column2
FROM table_name;
```

Example:

```
SELECT name, salary
FROM employees;
```

Select all columns:

```
SELECT *
FROM employees;
```

Use ***** only for quick testing. In interviews and production code, prefer explicit column names.

Aliases

Aliases rename columns or tables temporarily in the result.

```
SELECT name AS employee_name, salary AS monthly_salary
FROM employees;
```

AS is optional in many databases:

```
SELECT name employee_name
FROM employees;
```

Table aliases are very common:

```
SELECT e.name, e.salary
FROM employees e;
```

DISTINCT

DISTINCT removes duplicate rows from the result.

```
SELECT DISTINCT department_id
FROM employees;
```

For multiple columns, uniqueness is checked on the full combination:

```
SELECT DISTINCT department_id, job_title
FROM employees;
```

Expressions in SELECT

```
SELECT name, salary, salary * 12 AS annual_salary
FROM employees;
```

Common SQL Data Types

- **INT**, **BIGINT**: whole numbers
- **DECIMAL(p, s)**, **NUMERIC(p, s)**: exact decimal values, useful for money
- **FLOAT**, **DOUBLE**: approximate decimal values
- **CHAR(n)**: fixed-length string
- **VARCHAR(n)**: variable-length string
- **TEXT**: long text
- **DATE**: date only
- **TIME**: time only
- **DATETIME**, **TIMESTAMP**: date and time
- **BOOLEAN**: true/false, depending on DB support

2. WHERE Clause

WHERE filters rows before grouping or aggregation.

```
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Comparison Operators

=	equal
<>	not equal
!=	not equal, supported by many DBs
>	greater than
<	less than
>=	greater than or equal
<=	less than or equal

Examples:

```
SELECT *
FROM employees
WHERE department_id = 10;
```

```
SELECT *
FROM employees
WHERE salary >= 60000;
```

Logical Operators

```
SELECT *
FROM employees
WHERE department_id = 10 AND salary > 50000;
```

```
SELECT *
FROM employees
WHERE department_id = 10 OR department_id = 20;
```

```
SELECT *
FROM employees
WHERE NOT department_id = 10;
```

Use parentheses when mixing AND and OR.

```
SELECT *
FROM employees
```

```
WHERE department_id = 10
      OR (department_id = 20 AND salary > 70000);
```

AND has higher precedence than **OR**.

BETWEEN

BETWEEN includes both boundary values.

```
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 80000;
```

Equivalent to:

```
WHERE salary >= 40000 AND salary <= 80000
```

IN

IN checks whether a value exists in a list.

```
SELECT *
FROM employees
WHERE department_id IN (10, 20, 30);
```

Equivalent to:

```
WHERE department_id = 10
      OR department_id = 20
      OR department_id = 30
```

LIKE

LIKE is used for pattern matching.

```
SELECT *
FROM employees
WHERE name LIKE 'A%';
```

Common wildcards:

- **%**: zero or more characters

- `_`: exactly one character

Examples:

```
-- Names starting with A
WHERE name LIKE 'A%'

-- Names ending with n
WHERE name LIKE '%n'

-- Names containing "ar"
WHERE name LIKE '%ar%'

-- 5-letter names starting with A
WHERE name LIKE 'A_____'
```

NULL Handling

NULL means unknown or missing value. It is not equal to anything, not even another **NULL**.

Wrong:

```
WHERE manager_id = NULL
```

Correct:

```
WHERE manager_id IS NULL
```

```
WHERE manager_id IS NOT NULL
```

Three-Valued Logic

SQL conditions can evaluate to:

- **TRUE**
- **FALSE**
- **UNKNOWN**

Rows are returned only when the **WHERE** condition is **TRUE**.

Example:

```
SELECT *
FROM employees
```

```
WHERE bonus <> 10000;
```

Rows where `bonus` is `NULL` are not returned because `NULL <> 10000` is `UNKNOWN`.

COALESCE

`COALESCE` returns the first non-NULL value.

```
SELECT name, COALESCE(bonus, 0) AS bonus_amount
FROM employees;
```

3. ORDER BY

`ORDER BY` sorts the result.

```
SELECT name, salary
FROM employees
ORDER BY salary;
```

Default order is ascending.

```
ORDER BY salary ASC
```

Descending order:

```
ORDER BY salary DESC
```

Sort by multiple columns:

```
SELECT name, department_id, salary
FROM employees
ORDER BY department_id ASC, salary DESC;
```

This sorts by department first, then salary within each department.

ORDER BY With Aliases

```
SELECT name, salary * 12 AS annual_salary
FROM employees
```

```
ORDER BY annual_salary DESC;
```

NULL Sorting

Different databases handle **NULL** differently in sorting.

Some support:

```
ORDER BY salary DESC NULLS LAST;
```

If unsupported, use:

```
ORDER BY  
  CASE WHEN salary IS NULL THEN 1 ELSE 0 END,  
  salary DESC;
```

4. GROUP BY

GROUP BY groups rows so aggregate functions can be applied per group.

```
SELECT department_id, COUNT(*) AS employee_count  
FROM employees  
GROUP BY department_id;
```

Aggregate Functions

Common aggregate functions:

- **COUNT(*)**: counts rows
- **COUNT(column)**: counts non-NULL values in that column
- **SUM(column)**: total
- **AVG(column)**: average
- **MIN(column)**: minimum
- **MAX(column)**: maximum

Examples:

```
SELECT department_id, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department_id;
```

```
SELECT department_id, MIN(salary), MAX(salary)
FROM employees
GROUP BY department_id;
```

Important GROUP BY Rule

Every selected column must either:

1. Appear in the **GROUP BY**, or
2. Be inside an aggregate function.

Wrong:

```
SELECT department_id, name, AVG(salary)
FROM employees
GROUP BY department_id;
```

Correct:

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id;
```

GROUP BY Multiple Columns

```
SELECT department_id, job_title, COUNT(*) AS total
FROM employees
GROUP BY department_id, job_title;
```

This creates one group for every unique (**department_id**, **job_title**) pair.

COUNT Variants

```
SELECT COUNT(*) FROM employees;
```

Counts all rows.

```
SELECT COUNT(manager_id) FROM employees;
```

Counts rows where **manager_id** is not NULL.

```
SELECT COUNT(DISTINCT department_id) FROM employees;
```

Counts unique departments.

5. HAVING

HAVING filters groups after aggregation.

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Use **WHERE** for row-level filtering. Use **HAVING** for group-level filtering.

Example:

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
WHERE salary IS NOT NULL
GROUP BY department_id
HAVING AVG(salary) > 60000;
```

Here:

- **WHERE salary IS NOT NULL** filters rows before grouping.
- **HAVING AVG(salary) > 60000** filters grouped departments.

WHERE vs HAVING

```
-- Filters individual employees
WHERE salary > 50000

-- Filters departments based on average salary
HAVING AVG(salary) > 50000
```

6. Logical Query Execution Order

SQL is written in one order but logically processed in another.

Written order:

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
```

Logical execution order:

```
FROM
JOIN
WHERE
GROUP BY
HAVING
SELECT
DISTINCT
ORDER BY
LIMIT/OFFSET
```

This explains why a **SELECT** alias usually cannot be used in **WHERE**, because **WHERE** is processed before **SELECT**.

Wrong in many databases:

```
SELECT salary * 12 AS annual_salary
FROM employees
WHERE annual_salary > 700000;
```

Correct:

```
SELECT salary * 12 AS annual_salary
FROM employees
WHERE salary * 12 > 700000;
```

Or use a subquery/CTE.

7. Joins

Joins combine rows from multiple tables based on related columns.

Assume:

```
employees(employee_id, name, department_id, salary)
departments(department_id, department_name)
```

INNER JOIN

Returns only matching rows from both tables.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

If an employee has no matching department, that employee is not returned.

LEFT JOIN

Returns all rows from the left table and matching rows from the right table. If no match exists, right-side columns are **NULL**.

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id;
```

Useful for finding unmatched rows:

```
SELECT e.name
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

This finds employees without a valid department.

RIGHT JOIN

Returns all rows from the right table and matching rows from the left table.

```
SELECT e.name, d.department_name
FROM employees e
RIGHT JOIN departments d
    ON e.department_id = d.department_id;
```

Can usually be rewritten as a **LEFT JOIN** by swapping table order.

FULL OUTER JOIN

Returns all rows when there is a match in either table.

```
SELECT e.name, d.department_name
FROM employees e
FULL OUTER JOIN departments d
  ON e.department_id = d.department_id;
```

Not supported by MySQL directly. In MySQL, it can be simulated using **LEFT JOIN**, **RIGHT JOIN**, and **UNION**.

CROSS JOIN

Returns the Cartesian product of both tables.

```
SELECT e.name, d.department_name
FROM employees e
CROSS JOIN departments d;
```

If table A has 5 rows and table B has 4 rows, the result has 20 rows.

SELF JOIN

A table joins with itself.

Example: employee and manager from the same table.

```
SELECT e.name AS employee_name,
       m.name AS manager_name
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

Join Condition: ON vs WHERE

For **INNER JOIN**, filtering in **ON** or **WHERE** often produces the same result.

For **LEFT JOIN**, placement matters.

```
-- Keeps all employees, only matching active departments
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id
 AND d.active = 1;
```

```
-- Turns the LEFT JOIN into an INNER JOIN-like result
SELECT e.name, d.department_name
```

```
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id
WHERE d.active = 1;
```

The second query removes rows where the department side is **NULL**.

Common Join Mistakes

- Forgetting the join condition, causing a Cartesian product.
- Joining on the wrong column.
- Using **WHERE** incorrectly after **LEFT JOIN**.
- Selecting ambiguous column names without table aliases.
- Assuming **NULL = NULL** matches in joins. It does not.

8. Subqueries

A subquery is a query inside another query.

Subquery in WHERE

Find employees earning more than the average salary:

```
SELECT name, salary
FROM employees
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
);
```

Subquery With IN

Find employees working in IT or HR departments:

```
SELECT name
FROM employees
WHERE department_id IN (
  SELECT department_id
  FROM departments
  WHERE department_name IN ('IT', 'HR')
);
```

Subquery With EXISTS

EXISTS checks whether the subquery returns at least one row.

```
SELECT d.department_name
FROM departments d
WHERE EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department_id = d.department_id
);
```

This returns departments that have at least one employee.

NOT EXISTS

Find departments with no employees:

```
SELECT d.department_name
FROM departments d
WHERE NOT EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department_id = d.department_id
);
```

NOT EXISTS is often safer than **NOT IN** when **NULL** values are possible.

NOT IN and NULL Trap

If the subquery returns **NULL**, **NOT IN** may return no rows unexpectedly.

Risky:

```
SELECT name
FROM employees
WHERE department_id NOT IN (
    SELECT department_id
    FROM departments
);
```

Safer:

```
SELECT e.name
FROM employees e
WHERE NOT EXISTS (
    SELECT 1
    FROM departments d
```

```
WHERE d.department_id = e.department_id
);
```

Correlated Subquery

A correlated subquery depends on the outer query.

Find employees earning more than their department average:

```
SELECT e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e.department_id
);
```

The inner query runs logically for each outer row.

Subquery in FROM

Subqueries in **FROM** create derived tables.

```
SELECT dept_avg.department_id, dept_avg.avg_salary
FROM (
    SELECT department_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department_id
) dept_avg
WHERE dept_avg.avg_salary > 60000;
```

Subquery in SELECT

```
SELECT e.name,
       e.salary,
       (SELECT AVG(salary) FROM employees) AS company_avg_salary
FROM employees e;
```

Use carefully, because it may be inefficient depending on database optimization.

9. CTEs

CTE means Common Table Expression. It creates a temporary named result set for a query.

```
WITH dept_avg AS (  
  SELECT department_id, AVG(salary) AS avg_salary  
  FROM employees  
  GROUP BY department_id  
)  
SELECT *  
FROM dept_avg  
WHERE avg_salary > 60000;
```

CTEs improve readability, especially for multi-step queries.

CTE vs Subquery

Both can often solve the same problem.

Use a CTE when:

- The query has multiple logical steps.
- The same result is reused.
- Readability matters.
- You want to debug intermediate results.

Use a subquery when:

- The logic is short.
- It is used only once.

Multiple CTEs

```
WITH dept_count AS (  
  SELECT department_id, COUNT(*) AS employee_count  
  FROM employees  
  GROUP BY department_id  
)  
dept_avg AS (  
  SELECT department_id, AVG(salary) AS avg_salary  
  FROM employees  
  GROUP BY department_id  
)  
SELECT c.department_id, c.employee_count, a.avg_salary  
FROM dept_count c  
JOIN dept_avg a  
  ON c.department_id = a.department_id;
```

Recursive CTE

Recursive CTEs are used for hierarchical data such as employee-manager trees, category trees, or graph traversal.

Example table:

```
employees(employee_id, name, manager_id)
```

Find hierarchy starting from the CEO:

```
WITH RECURSIVE employee_tree AS (  
  SELECT employee_id, name, manager_id, 1 AS level  
  FROM employees  
  WHERE manager_id IS NULL  
  
  UNION ALL  
  
  SELECT e.employee_id, e.name, e.manager_id, et.level + 1  
  FROM employees e  
  JOIN employee_tree et  
    ON e.manager_id = et.employee_id  
)  
SELECT *  
FROM employee_tree  
ORDER BY level, employee_id;
```

Note: exact syntax varies across databases. SQL Server uses recursive CTEs but does not use the **RECURSIVE** keyword.

10. Window Functions

Window functions perform calculations across related rows without collapsing rows like **GROUP BY**.

Basic syntax:

```
function_name() OVER (  
  PARTITION BY column  
  ORDER BY column  
)
```

GROUP BY vs Window Function

GROUP BY reduces rows.

```
SELECT department_id, AVG(salary)  
FROM employees  
GROUP BY department_id;
```

One row per department.

Window function keeps original rows.

```
SELECT name,  
       department_id,  
       salary,  
       AVG(salary) OVER (PARTITION BY department_id) AS dept_avg_salary  
FROM employees;
```

One row per employee, with department average shown beside each employee.

ROW_NUMBER

Assigns a unique sequence number in each partition.

```
SELECT name,  
       department_id,  
       salary,  
       ROW_NUMBER() OVER (  
         PARTITION BY department_id  
         ORDER BY salary DESC  
       ) AS rn  
FROM employees;
```

Find highest-paid employee in each department:

```
WITH ranked AS (  
  SELECT name,  
         department_id,  
         salary,  
         ROW_NUMBER() OVER (  
           PARTITION BY department_id  
           ORDER BY salary DESC  
         ) AS rn  
  FROM employees  
)  
SELECT *  
FROM ranked  
WHERE rn = 1;
```

If there is a tie, `ROW_NUMBER` still gives different numbers.

RANK

Gives the same rank for ties, but skips ranks after ties.

```
SELECT name,  
       salary,  
       RANK() OVER (ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

If salaries are:

```
100, 90, 90, 80
```

Ranks are:

```
1, 2, 2, 4
```

DENSE_RANK

Gives the same rank for ties, without skipping ranks.

```
SELECT name,  
       salary,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

For:

```
100, 90, 90, 80
```

Dense ranks are:

```
1, 2, 2, 3
```

NTILE

Divides rows into buckets.

```
SELECT name,  
       salary,  
       NTILE(4) OVER (ORDER BY salary DESC) AS salary_quartile  
FROM employees;
```

This divides employees into 4 salary groups.

LAG

Gets a value from a previous row.

```
SELECT employee_id,  
       salary,  
       LAG(salary) OVER (ORDER BY employee_id) AS previous_salary  
FROM employees;
```

Useful for comparing current row with previous row.

LEAD

Gets a value from a future row.

```
SELECT employee_id,  
       salary,  
       LEAD(salary) OVER (ORDER BY employee_id) AS next_salary  
FROM employees;
```

Running Total

```
SELECT order_date,  
       amount,  
       SUM(amount) OVER (  
         ORDER BY order_date  
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
       ) AS running_total  
FROM orders;
```

Moving Average

```
SELECT order_date,  
       amount,  
       AVG(amount) OVER (  
         ORDER BY order_date  
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
       ) AS moving_avg_3_rows  
FROM orders;
```

FIRST_VALUE and LAST_VALUE

```
SELECT name,  
       department_id,  
       salary,  
       FIRST_VALUE(name) OVER (  
         PARTITION BY department_id  
         ORDER BY salary DESC  
       ) AS highest_paid_employee  
FROM employees;
```

Be careful with `LAST_VALUE`; default window frames can produce surprising results. Often specify the frame:

```
LAST_VALUE(name) OVER (  
  PARTITION BY department_id  
  ORDER BY salary DESC  
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
)
```

Window Frame

The frame decides which rows are visible to the window calculation.

Common frame:

```
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

Meaning: from the first row of the partition to the current row.

Another frame:

```
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
```

Meaning: previous row, current row, and next row.

Top N Per Group

Find top 3 salaries per department:

```
WITH ranked AS (  
  SELECT name,  
         department_id,  
         salary,  
         DENSE_RANK() OVER (  
           PARTITION BY department_id  
           ORDER BY salary DESC  
         )  
)
```

```
    ) AS rnk
  FROM employees
)
SELECT *
FROM ranked
WHERE rnk <= 3;
```

Use `DENSE_RANK` if ties should be included. Use `ROW_NUMBER` if exactly 3 rows per department are needed.

11. LIMIT, OFFSET, FETCH

Different databases use different syntax.

MySQL/PostgreSQL/SQLite:

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

With offset:

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 5 OFFSET 10;
```

SQL Server:

```
SELECT TOP 5 *
FROM employees
ORDER BY salary DESC;
```

Standard SQL style:

```
SELECT *
FROM employees
ORDER BY salary DESC
FETCH FIRST 5 ROWS ONLY;
```

12. Set Operations

Set operations combine results of multiple queries.

UNION

Combines results and removes duplicates.

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;
```

UNION ALL

Combines results and keeps duplicates.

```
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
```

UNION ALL is usually faster because it does not remove duplicates.

INTERSECT

Returns rows common to both queries.

```
SELECT city FROM customers
INTERSECT
SELECT city FROM suppliers;
```

EXCEPT / MINUS

Returns rows from the first query that are not in the second.

```
SELECT city FROM customers
EXCEPT
SELECT city FROM suppliers;
```

Oracle uses **MINUS** instead of **EXCEPT**.

Rules:

- Both queries must return the same number of columns.
- Corresponding columns should have compatible data types.
- **ORDER BY** usually appears at the end.

13. CASE Expression

CASE is SQL's conditional expression.

```
SELECT name,
       salary,
       CASE
         WHEN salary >= 100000 THEN 'High'
         WHEN salary >= 50000 THEN 'Medium'
         ELSE 'Low'
       END AS salary_band
FROM employees;
```

Useful inside aggregation:

```
SELECT department_id,
       SUM(CASE WHEN salary > 50000 THEN 1 ELSE 0 END) AS high_salary_count
FROM employees
GROUP BY department_id;
```

14. Important Interview Query Patterns

Second Highest Salary

Using subquery:

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (
  SELECT MAX(salary)
  FROM employees
);
```

Using window function:

```
WITH ranked AS (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
)
SELECT salary
FROM ranked
WHERE rnk = 2;
```

Nth Highest Salary

```
WITH ranked AS (  
    SELECT salary,  
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk  
    FROM employees  
)  
SELECT salary  
FROM ranked  
WHERE rnk = 5;
```

Duplicate Records

Find duplicate emails:

```
SELECT email, COUNT(*) AS total  
FROM users  
GROUP BY email  
HAVING COUNT(*) > 1;
```

Delete duplicates usually requires a primary key and database-specific syntax. The safe idea is to keep one row and remove rows with higher/lower ids.

Employees With Salary Greater Than Department Average

```
SELECT e.name, e.salary, e.department_id  
FROM employees e  
WHERE e.salary > (  
    SELECT AVG(e2.salary)  
    FROM employees e2  
    WHERE e2.department_id = e.department_id  
);
```

Window function version:

```
WITH marked AS (  
    SELECT name,  
           salary,  
           department_id,  
           AVG(salary) OVER (PARTITION BY department_id) AS dept_avg  
    FROM employees  
)  
SELECT *  
FROM marked  
WHERE salary > dept_avg;
```

Departments With More Than 5 Employees

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Customers Who Never Ordered

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Or:

```
SELECT c.customer_id, c.name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.customer_id
);
```

Highest Salary in Each Department

```
WITH ranked AS (
  SELECT name,
         department_id,
         salary,
         DENSE_RANK() OVER (
           PARTITION BY department_id
           ORDER BY salary DESC
         ) AS rnk
  FROM employees
)
SELECT *
FROM ranked
WHERE rnk = 1;
```

Running Total of Sales

```
SELECT order_date,
       amount,
       SUM(amount) OVER (
         ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM orders;
```

Find Consecutive Records

Example: find users who logged in on consecutive days. Exact syntax varies by database, but the idea often uses `LAG`.

```
WITH marked AS (
  SELECT user_id,
         login_date,
         LAG(login_date) OVER (
           PARTITION BY user_id
           ORDER BY login_date
         ) AS previous_login_date
  FROM logins
)
SELECT *
FROM marked
WHERE login_date = previous_login_date + INTERVAL '1 day';
```

In MySQL, date arithmetic may use:

```
DATE_ADD(previous_login_date, INTERVAL 1 DAY)
```

15. SQL Clause Summary

```
SELECT columns
FROM table
JOIN another_table
  ON join_condition
WHERE row_filter
GROUP BY grouping_columns
HAVING group_filter
ORDER BY sort_columns
LIMIT number;
```

Meaning:

- **SELECT**: choose columns or expressions.
- **FROM**: choose source table.
- **JOIN**: combine tables.
- **WHERE**: filter rows.
- **GROUP BY**: group rows.
- **HAVING**: filter groups.
- **ORDER BY**: sort final result.
- **LIMIT**: restrict number of rows.

16. Performance Basics for Placements

Indexes

Indexes help the database find rows faster.

Common columns to index:

- Primary keys
- Foreign keys
- Columns frequently used in **WHERE**
- Columns frequently used in **JOIN**
- Columns frequently used in **ORDER BY**

Example:

```
CREATE INDEX idx_employees_department_id
ON employees(department_id);
```

Index Trade-Off

Indexes improve read performance but slow down writes because the index must also be updated during **INSERT**, **UPDATE**, and **DELETE**.

Avoid Functions on Indexed Columns in WHERE

May prevent index usage:

```
WHERE YEAR(order_date) = 2026
```

Better:

```
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01'
```

SELECT Only Needed Columns

Avoid:

```
SELECT *
```

Prefer:

```
SELECT employee_id, name, salary
```

Use EXISTS for Existence Checks

Often better:

```
WHERE EXISTS (  
  SELECT 1  
  FROM orders o  
  WHERE o.customer_id = c.customer_id  
)
```

Instead of counting:

```
WHERE (  
  SELECT COUNT(*)  
  FROM orders o  
  WHERE o.customer_id = c.customer_id  
) > 0
```

Read the Query Plan

Databases provide query plans:

```
EXPLAIN SELECT ...
```

or:

```
EXPLAIN ANALYZE SELECT ...
```

Important terms:

- Full table scan: reads many/all rows.
- Index scan/seek: uses index.

- Nested loop join: loops through one table and probes another.
- Hash join: builds a hash table for joining.
- Sort: database must sort data, can be expensive.

17. Normalization Quick Revision

Although this file focuses on SQL queries, DBMS interviews often connect SQL with normalization.

1NF

- Each column contains atomic values.
- No repeating groups or arrays in a single column.

2NF

- Must be in 1NF.
- No partial dependency on part of a composite key.

3NF

- Must be in 2NF.
- No transitive dependency.
- Non-key columns should depend only on the key.

BCNF

- Stronger version of 3NF.
- For every functional dependency $X \rightarrow Y$, X should be a super key.

18. Transactions Quick Revision

Transaction properties are called ACID.

Atomicity

All operations happen completely or none happen.

Consistency

Transaction takes the database from one valid state to another valid state.

Isolation

Concurrent transactions should not interfere incorrectly.

Durability

Once committed, data remains saved even after failure.

Example:

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 2;

COMMIT;
```

If something fails:

```
ROLLBACK;
```

19. Common Interview Gotchas

- **WHERE** filters rows; **HAVING** filters groups.
- **COUNT(*)** counts rows; **COUNT(column)** ignores NULLs.
- **DISTINCT** applies to the whole selected row, not one column separately.
- **NULL** cannot be checked using **= NULL**; use **IS NULL**.
- **NOT IN** can behave unexpectedly if the subquery returns NULL.
- **LEFT JOIN** can accidentally become an inner join if right-table filters are placed in **WHERE**.
- **GROUP BY** reduces rows; window functions keep rows.
- **ROW_NUMBER**, **RANK**, and **DENSE_RANK** handle ties differently.
- **UNION** removes duplicates; **UNION ALL** keeps duplicates.
- **BETWEEN** is inclusive.
- SQL logical execution order is different from written order.
- Always use table aliases when joining multiple tables.
- Always think about duplicates when joining one-to-many tables.

20. Fast Revision Checklist

Before an SQL interview, make sure you can write queries for:

- Selecting specific columns.
- Filtering with **WHERE**, **IN**, **BETWEEN**, **LIKE**, and **IS NULL**.
- Sorting with **ORDER BY**.
- Grouping with **GROUP BY**.
- Filtering groups with **HAVING**.
- Inner, left, right, full, cross, and self joins.
- Finding unmatched rows with **LEFT JOIN ... IS NULL**.
- Subqueries with **IN**, **EXISTS**, and scalar comparisons.
- Correlated subqueries.
- CTEs and recursive CTEs.

- **ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD.**
- Top N per group.
- Second highest and nth highest salary.
- Duplicate detection.
- Running totals.
- Difference between **WHERE** and **HAVING**.
- Difference between **GROUP BY** and window functions.
- NULL behavior.
- Basic index and query performance concepts.

21. Mini Practice Schema

Use this schema mentally for practice:

```
students(student_id, name, branch, cgpa)
courses(course_id, course_name, credits)
enrollments(student_id, course_id, marks)
employees(employee_id, name, department_id, manager_id, salary, joining_date)
departments(department_id, department_name, location)
customers(customer_id, name, city)
orders(order_id, customer_id, order_date, amount)
```

Practice questions:

1. Find students with CGPA greater than 8.
2. Find average marks per course.
3. Find courses with more than 50 enrolled students.
4. Find students who are not enrolled in any course.
5. Find the highest paid employee in each department.
6. Find employees earning more than their manager.
7. Find departments with no employees.
8. Find customers who placed more than 3 orders.
9. Find the second highest order amount.
10. Find running total of order amount by date.
11. Rank students by marks within each course.
12. Find employees who joined in the last 30 days.
13. Find duplicate customer names.
14. Find the top 3 customers by total purchase amount.
15. Find each employee's salary and department average salary.

22. Practice Answers

1. Students With CGPA Greater Than 8

```
SELECT student_id, name, branch, cgpa
FROM students
```

```
WHERE cgpa > 8;
```

2. Average Marks Per Course

```
SELECT course_id, AVG(marks) AS avg_marks
FROM enrollments
GROUP BY course_id;
```

3. Courses With More Than 50 Students

```
SELECT course_id, COUNT(*) AS student_count
FROM enrollments
GROUP BY course_id
HAVING COUNT(*) > 50;
```

4. Students Not Enrolled in Any Course

```
SELECT s.student_id, s.name
FROM students s
LEFT JOIN enrollments e
  ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

5. Highest Paid Employee in Each Department

```
WITH ranked AS (
  SELECT employee_id,
         name,
         department_id,
         salary,
         DENSE_RANK() OVER (
           PARTITION BY department_id
           ORDER BY salary DESC
         ) AS rnk
  FROM employees
)
SELECT *
FROM ranked
WHERE rnk = 1;
```

6. Employees Earning More Than Their Manager

```
SELECT e.name AS employee_name,  
       e.salary AS employee_salary,  
       m.name AS manager_name,  
       m.salary AS manager_salary  
FROM employees e  
JOIN employees m  
  ON e.manager_id = m.employee_id  
WHERE e.salary > m.salary;
```

7. Departments With No Employees

```
SELECT d.department_id, d.department_name  
FROM departments d  
LEFT JOIN employees e  
  ON d.department_id = e.department_id  
WHERE e.employee_id IS NULL;
```

8. Customers With More Than 3 Orders

```
SELECT customer_id, COUNT(*) AS order_count  
FROM orders  
GROUP BY customer_id  
HAVING COUNT(*) > 3;
```

9. Second Highest Order Amount

```
WITH ranked AS (  
  SELECT amount,  
         DENSE_RANK() OVER (ORDER BY amount DESC) AS rnk  
  FROM orders  
)  
SELECT amount  
FROM ranked  
WHERE rnk = 2;
```

10. Running Total of Orders by Date

```
SELECT order_date,  
       amount,  
       SUM(amount) OVER (  
         ORDER BY order_date  
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

```
    ) AS running_total  
FROM orders;
```

11. Rank Students by Marks Within Each Course

```
SELECT student_id,  
       course_id,  
       marks,  
       DENSE_RANK() OVER (  
         PARTITION BY course_id  
         ORDER BY marks DESC  
       ) AS course_rank  
FROM enrollments;
```

12. Employees Who Joined in the Last 30 Days

PostgreSQL style:

```
SELECT *  
FROM employees  
WHERE joining_date >= CURRENT_DATE - INTERVAL '30 days';
```

MySQL style:

```
SELECT *  
FROM employees  
WHERE joining_date >= DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY);
```

13. Duplicate Customer Names

```
SELECT name, COUNT(*) AS total  
FROM customers  
GROUP BY name  
HAVING COUNT(*) > 1;
```

14. Top 3 Customers by Total Purchase Amount

```
SELECT customer_id, SUM(amount) AS total_amount  
FROM orders  
GROUP BY customer_id
```

```
ORDER BY total_amount DESC
LIMIT 3;
```

SQL Server:

```
SELECT TOP 3 customer_id, SUM(amount) AS total_amount
FROM orders
GROUP BY customer_id
ORDER BY total_amount DESC;
```

15. Employee Salary With Department Average

```
SELECT name,
       department_id,
       salary,
       AVG(salary) OVER (PARTITION BY department_id) AS dept_avg_salary
FROM employees;
```

23. Final Mental Model

When solving SQL problems in interviews:

1. Identify the base table.
2. Decide if another table is needed.
3. Choose the correct join.
4. Apply row filters using **WHERE**.
5. Decide if grouping is needed.
6. Use aggregate functions if the result is per group.
7. Use **HAVING** for aggregate filters.
8. Use window functions if you need ranking, running totals, or group stats while keeping individual rows.
9. Sort only at the end.
10. Check NULLs and duplicates.

The biggest SQL skill is not memorizing syntax. It is knowing the shape of the result: one row per employee, one row per department, one row per customer, or one row per group. Once that is clear, the query becomes much easier.